Tecnologías IoT para la Industria 4.0 · Bloque 0

Práctica D5 · Despliega el entorno del curso

Despliegue automatizado de Mosquitto, Node-RED, InfluxDB y Grafana con un script

Alumno	Manu Martín
Curso	2.º DAM
Fecha	31 de octubre de 2026
Objetivo	Desplegar de forma automatizada el entorno que se usará durante todo el módulo
Criterios	CE6.2
Entrega	Capturas del despliegue, de `docker compose ps` y respuestas (carpeta `05-entorno-iot`)
Entorno	Windows 11, Docker Desktop (motor 29.8.2, Compose 5.6.0), PowerShell

Índice

1-2. Primera ejecución, claves nuevas y despliegue 3
3-4. Interfaces web, credenciales y data source 3
5. Copia de seguridad 5
6. down y despliegue de nuevo: los datos siguen 6
7. Reinicio 7
Comprueba que funciona 7
Preguntas 8
Retos 9
Estado final 10

La carpeta traía `desplegar.ps1` para Windows, pero las copias de seguridad solo estaban en bash. Se han escrito `copia.ps1` y `restaurar.ps1`, que hacen lo mismo que `copia.sh` y `restaurar.sh`, para trabajar todo desde PowerShell.

1-2. Primera ejecución, claves nuevas y despliegue

La primera vez no hay `.env`: el script lo crea a partir de `.env.example` y se detiene. Después se sustituyeron los tres valores `cambia-…` por claves y un token aleatorios. En las capturas aparecen ocultos con asteriscos, porque nunca deben salir en la documentación. Con el `.env` listo, el script descarga las cuatro imágenes (unos 5 minutos la primera vez), crea red y volúmenes, espera a que los cuatro servicios estén healthy y muestra las direcciones.

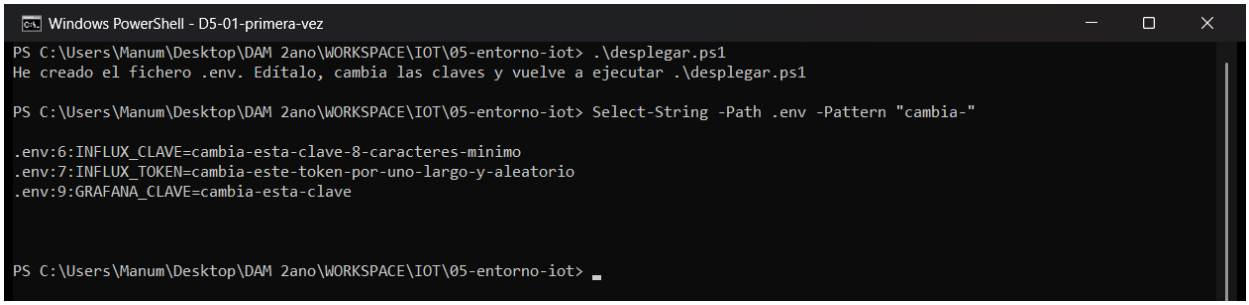


Figura 1. *Primera ejecución: el script crea el .env y se para.*

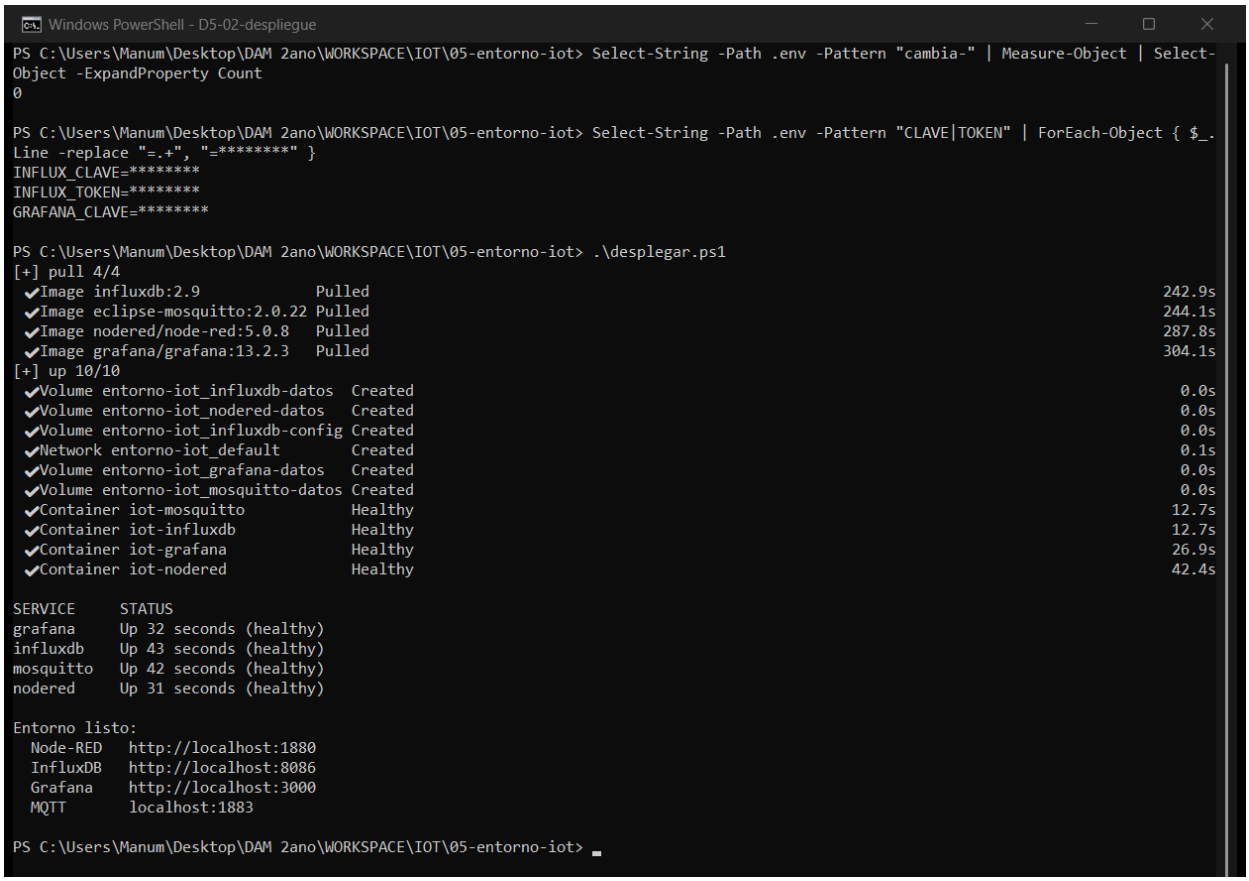


Figura 2. *Con las claves cambiadas, despliegue completo: cuatro servicios healthy.*

3-4. Interfaces web, credenciales y data source

Las tres interfaces responden: Node-RED (aquí se responde «No, do not enable notifications»), InfluxDB y Grafana. Las credenciales del `.env` se comprobaron desde PowerShell, leyendo el `.env` sin mostrarlo. InfluxDB tiene el bucket `sensores` creado en el primer arranque, y la API de Grafana, con el usuario y la

clave del `.env`, muestra el data source **«InfluxDB aula»**, que nadie ha creado a mano. Su comprobación de salud responde «datasource is working. 3 buckets found».

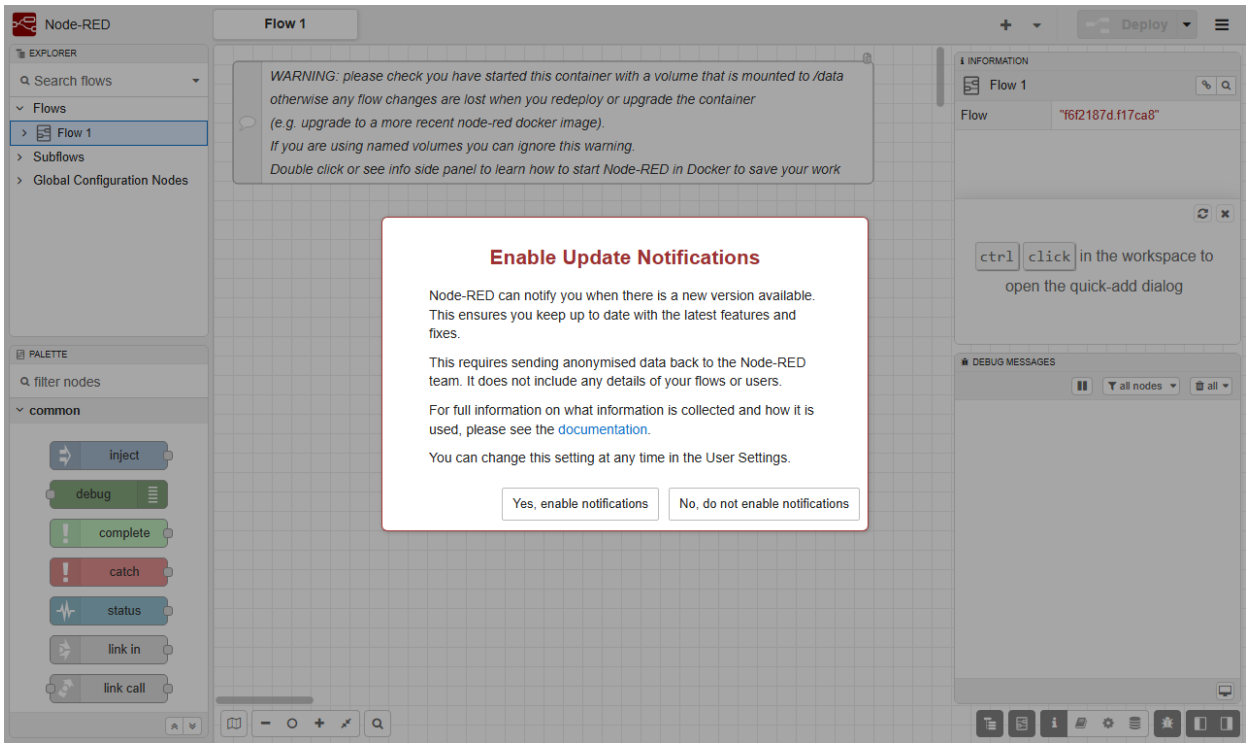


Figura 3. *Node-RED en http://localhost:1880, con el aviso de notificaciones.*

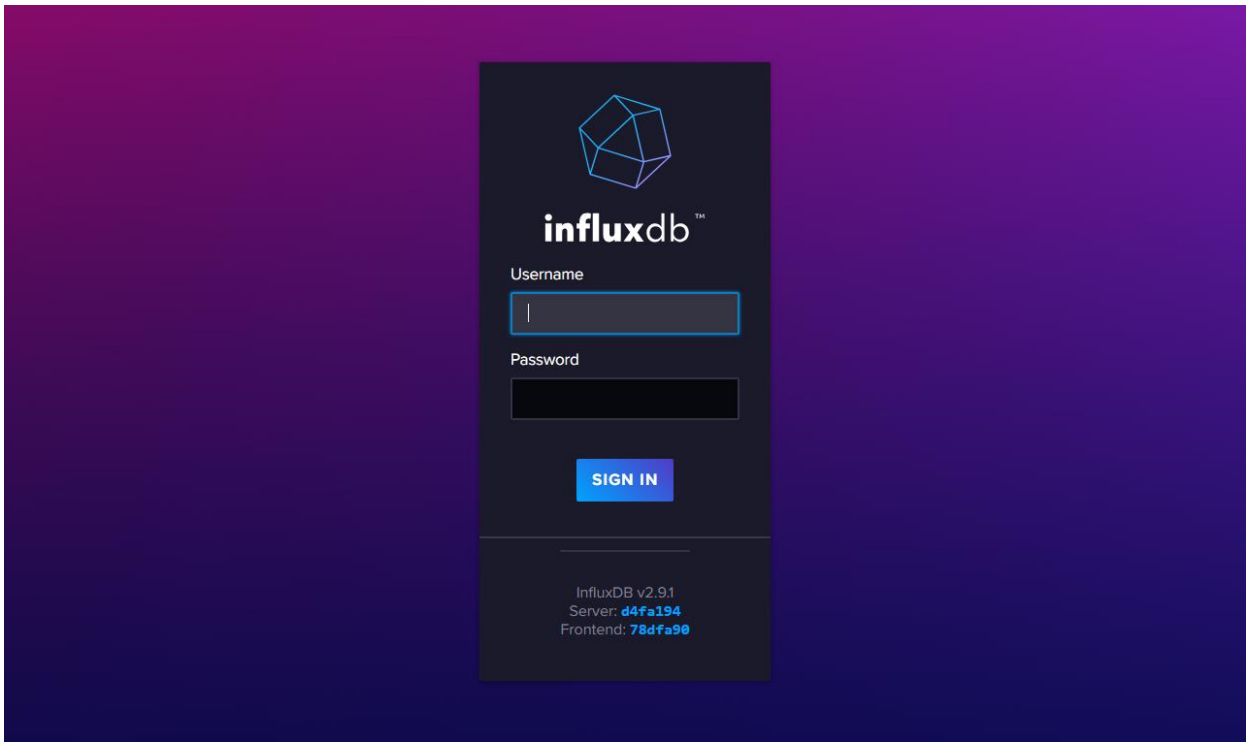


Figura 4. *InfluxDB en http://localhost:8086.*

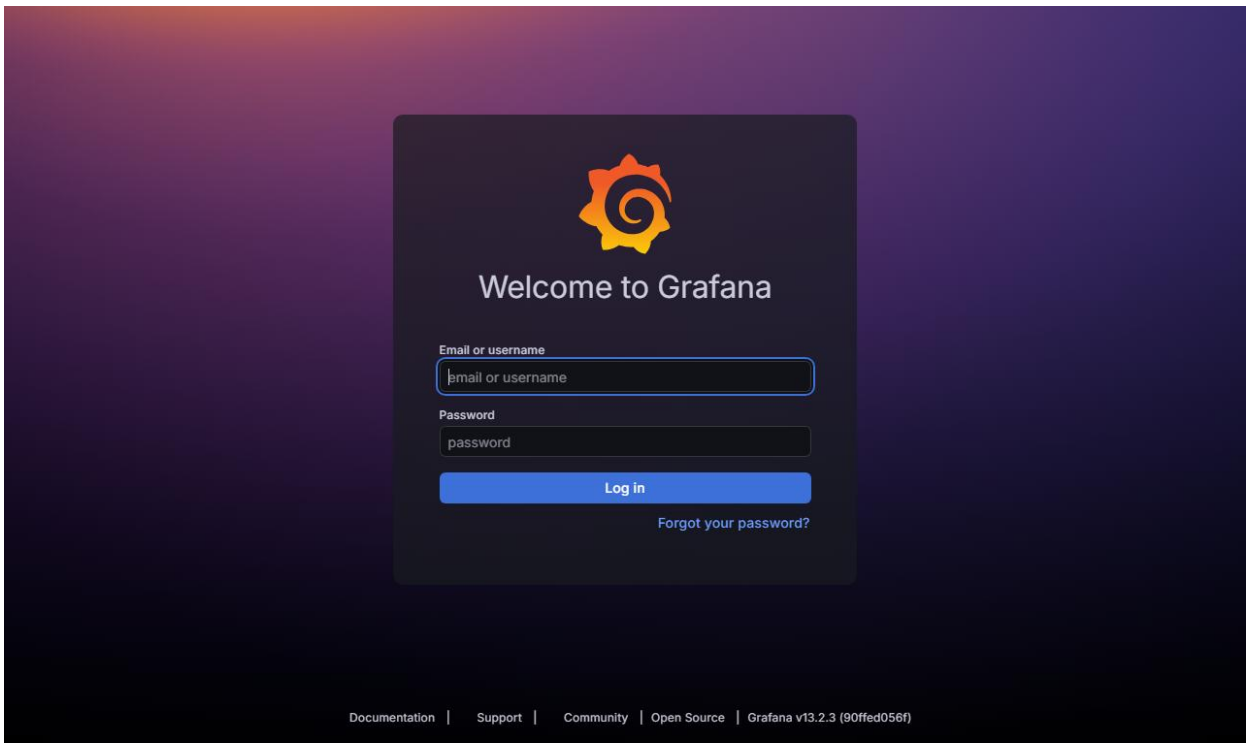


Figura 5. *Grafana en http://localhost:3000.*

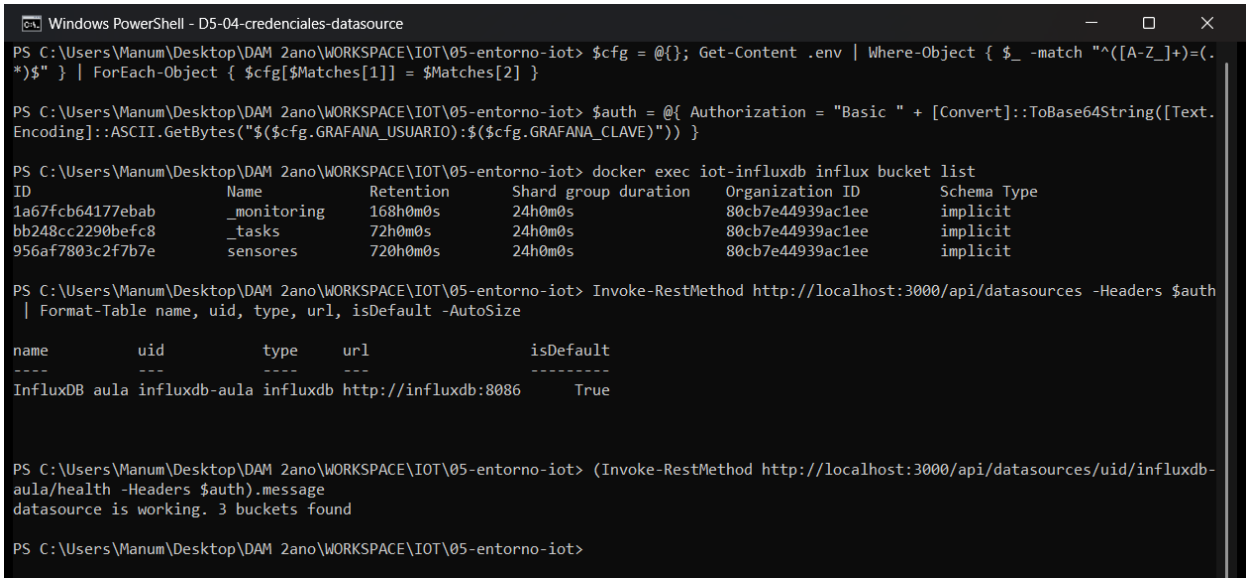


Figura 6. *Bucket sensores en InfluxDB y data source «InfluxDB aula» aprovisionado y funcionando.*

5. Copia de seguridad

Antes de la copia se escribió en InfluxDB un dato de prueba (`temperatura,sensor=prueba valor=21.5`) para poder comprobar después que los datos sobreviven. `copia.ps1` crea un fichero `.tar.gz` por volumen en `copias\`, usando un contenedor `alpine` de usar y tirar que monta el volumen en solo lectura.

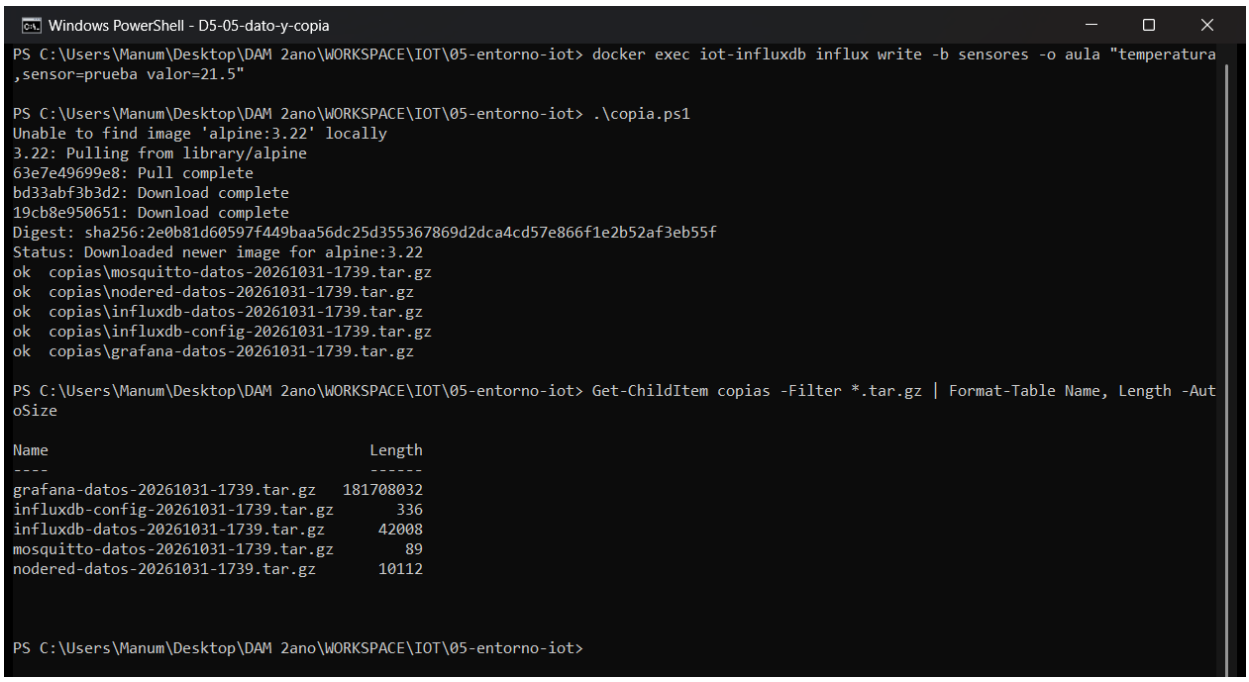


Figura 7. *Dato de prueba y copia: un fichero por volumen.*

6. down y despliegue de nuevo: los datos siguen

`docker compose down` borra los contenedores y la red, pero no los volúmenes. Tras volver a desplegar, una consulta Flux a la API de InfluxDB devuelve el dato de prueba escrito antes.

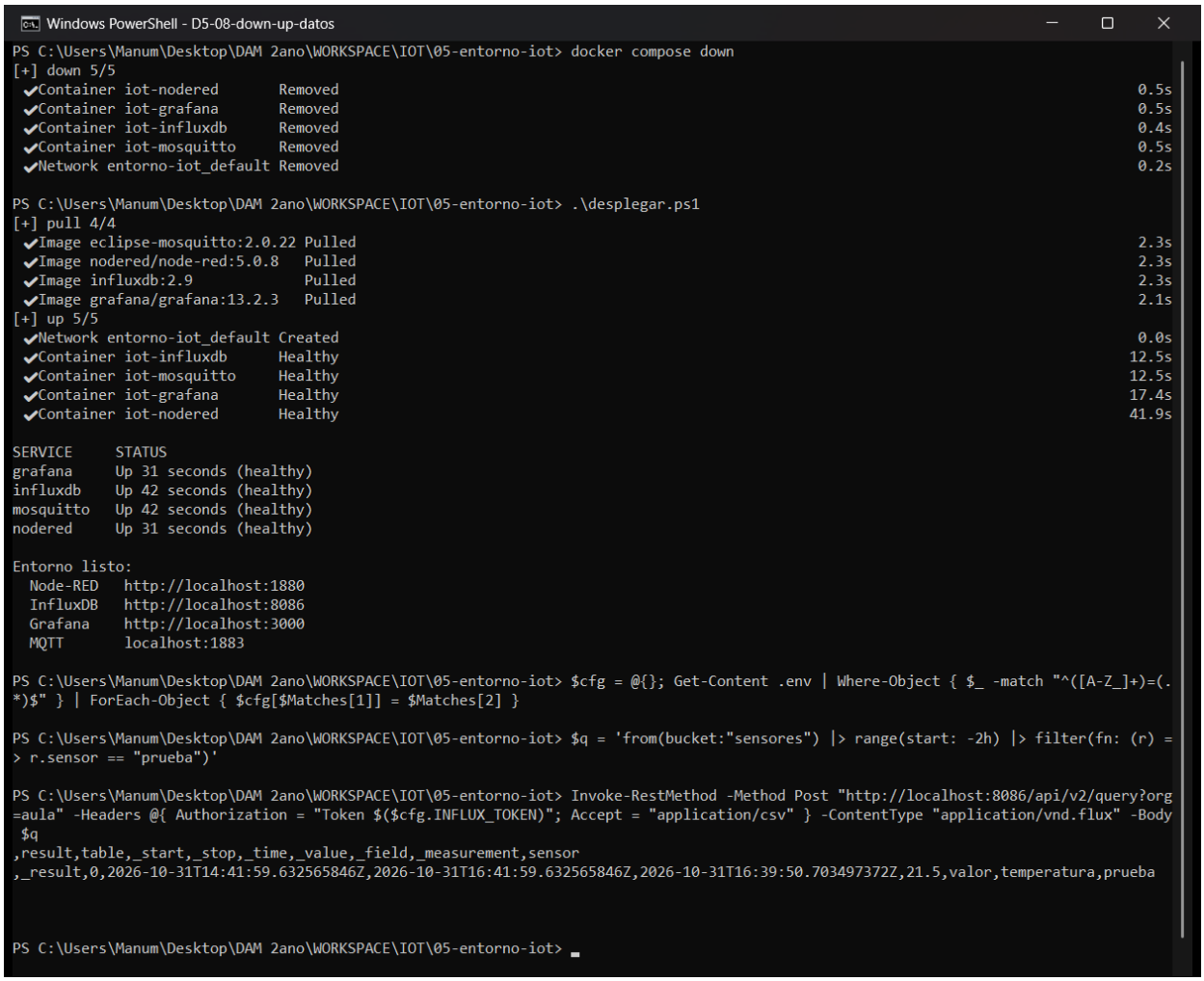


Figura 8. *Tras down y desplegar, el dato de prueba sigue en InfluxDB.*

7. Reinicio

En lugar de reiniciar el ordenador se reinició Docker Desktop (`docker desktop restart`), que tiene el mismo efecto sobre los contenedores. Gracias a `restart: unless-stopped`, los cuatro servicios vuelven a arrancar solos y quedan healthy sin ejecutar ningún comando de Compose. Para que vuelvan tras reiniciar el ordenador, Docker Desktop tiene que estar configurado para iniciarse con Windows.

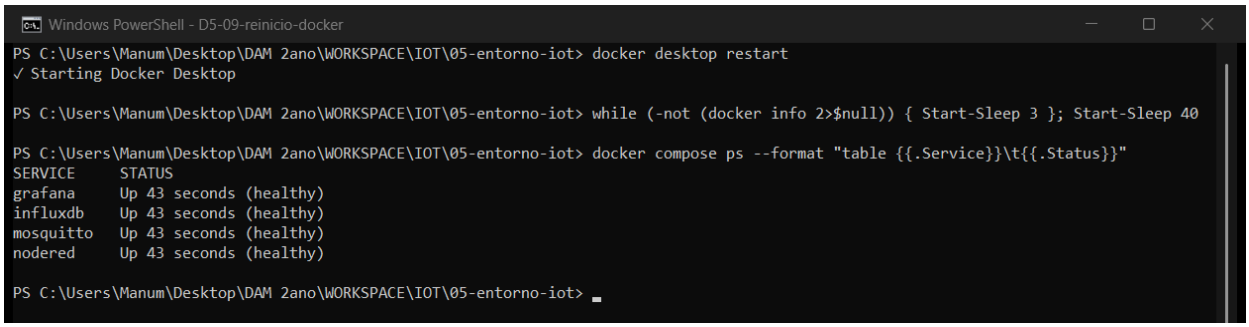


Figura 9. *Tras reiniciar Docker Desktop, los servicios vuelven solos.*

Comprueba que funciona

- **El script termina con los cuatro servicios healthy y muestra sus direcciones:** sí (Figura 2).
- **Ejecutarlo por segunda vez no cambia nada ni da errores:** sí, todos los contenedores siguen igual, «Up 3 minutes» (Figura 10).
- **Parar un servicio y volver a desplegar solo toca ese contenedor:** sí, solo `iot-grafana` aparece como `Started`; el resto, `Running` (Figura 11).
- **En la carpeta copias hay un fichero por volumen:** sí, cinco ficheros (Figura 7).

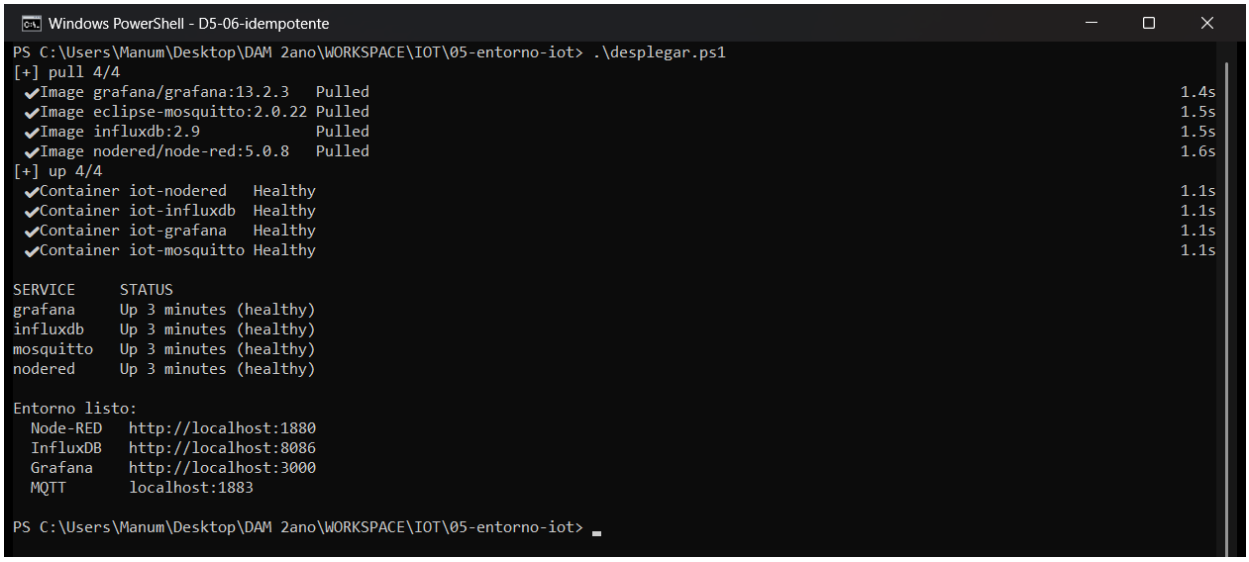


Figura 10. *Segunda ejecución: no recrea nada.*

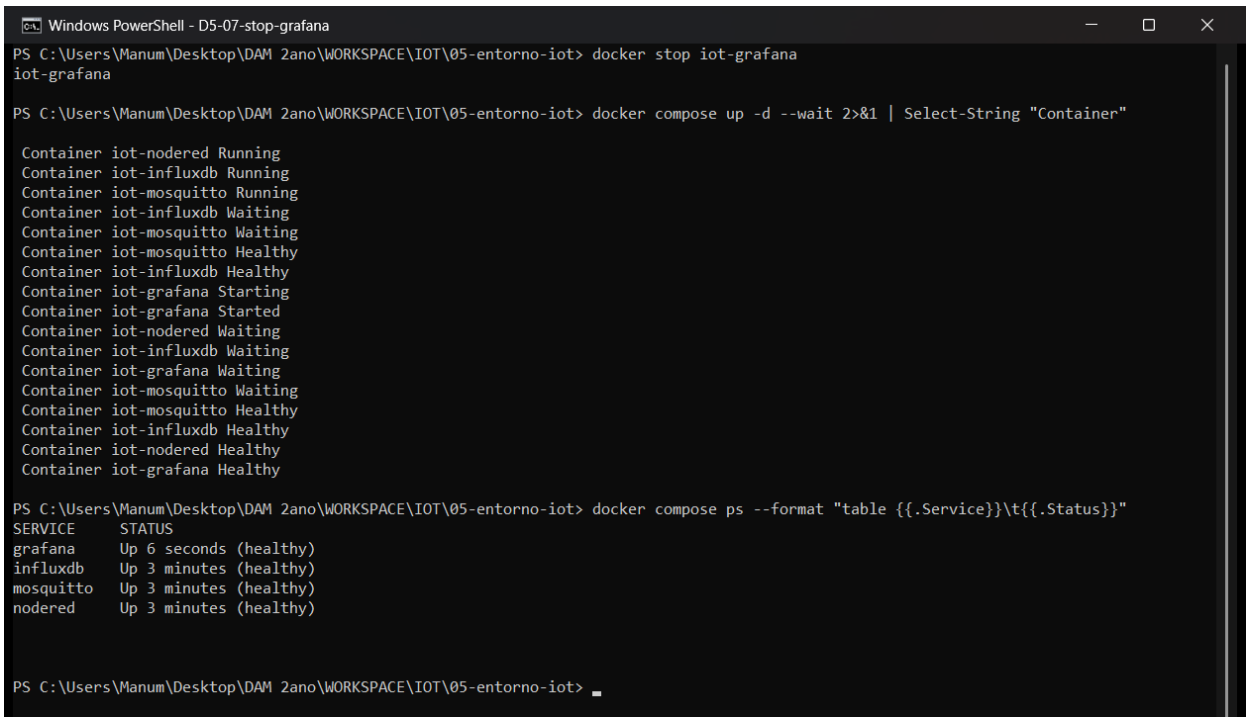


Figura 11. *Con Grafana parada, el despliegue solo arranca ese contenedor.*

Preguntas

¿Qué pasaría si el script no comprobara los valores cambia-…? El entorno arrancaría con las claves de ejemplo, que están publicadas en el repositorio y las conoce cualquiera: InfluxDB y Grafana quedarían accesibles con credenciales públicas. Además, como esas claves solo se aplican en el primer arranque, quedarían guardadas en los volúmenes. Cambiarlas después obligaría a hacerlo a mano dentro de cada servicio o a borrar los datos con `down -v`.

¿Por qué la contraseña de InfluxDB solo se aplica en el primer arranque? ¿Cómo cambiarla después? Las variables `DOCKER_INFLUXDB_INIT_*` solo actúan en modo `setup`, cuando el volumen está vacío. A partir de ahí el usuario ya existe dentro del volumen y la imagen las ignora. Para cambiarla hay que hacerlo en el propio InfluxDB, con `docker exec -it iot-influxdb influx user password -n admin` o desde su interfaz web, y actualizar `.env` para que coincida. La alternativa, `docker compose down -v`, borra todos los datos.

Las cinco piezas del apartado 0.12 en el entorno:

Pieza	Dónde está
Compose con espera	`docker compose up -d --wait` en `desplegar.ps1`, y `depends_on` con `condition: service_healthy` (nodered → mosquitto, grafana → influxdb)
Healthchecks	`healthcheck` de mosquitto (`mosquitto_sub`), influxdb (`influx ping`) y grafana (`/api/health`); Node-RED trae el suyo en la imagen
Inicialización por variables	`DOCKER_INFLUXDB_INIT_*`: usuario, organización `aula`, bucket `sensores` y token
Aprovisionamiento por ficheros	`./grafana/provisioning:/etc/grafana/provisioning:ro` con `datasources/influxdb.yaml`
Script de despliegue	`desplegar.ps1` / `desplegar.sh`

Retos

Restauración: se borró el volumen de Grafana y se recuperó con `restaurar.ps1` a partir de la copia. Grafana arranca healthy con su base de datos y el data source. El aviso amarillo de Compose aparece porque el volumen lo recreó el script y no Compose; es inofensivo.

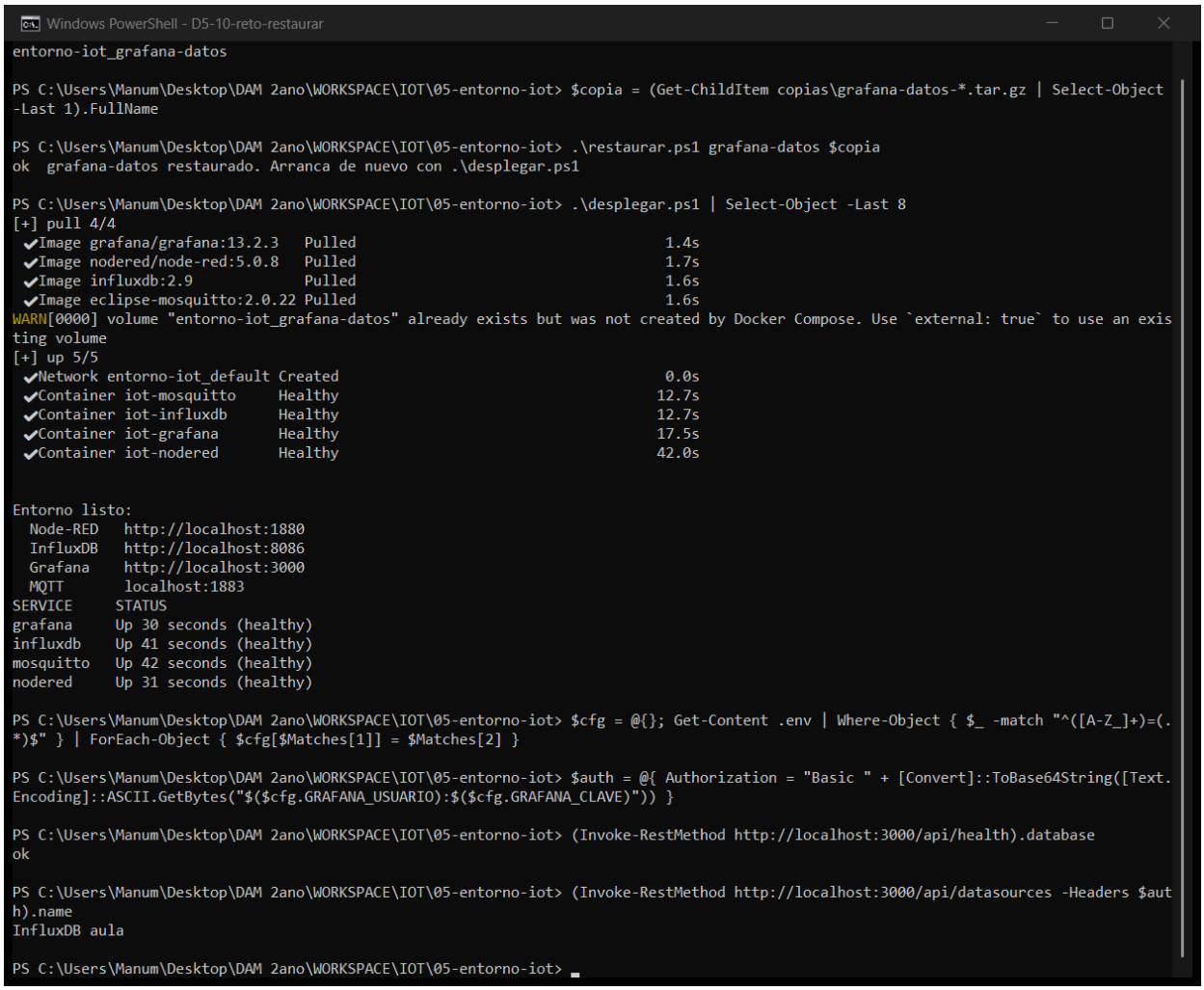


Figura 12. *Volumen de Grafana borrado y restaurado desde la copia.*

Aviso de puertos ocupados: `desplegar.ps1` comprueba ahora, antes de arrancar, si algún puerto del `.env` lo usa otro programa e indica cuál. Si el puerto lo ocupa nuestro propio servicio en marcha, no avisa, para que el redespliegue siga siendo idempotente. Para probarlo se ocupó el puerto 1880 desde PowerShell.

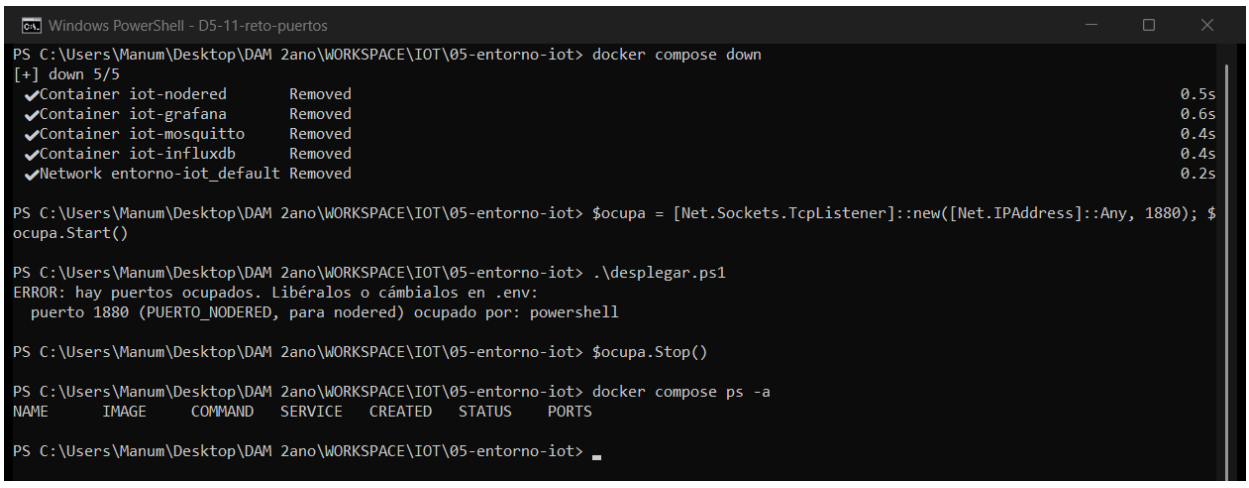


Figura 13. *El script detecta que el puerto 1880 está ocupado y no arranca.*

Microservicio como quinto servicio: no se ha hecho; el microservicio de D4 sigue desplegándose por separado desde `04-springboot`.

Estado final

`05-entorno-iot` incluye `copia.ps1`, `restaurar.ps1` y `desplegar.ps1` ampliado (aviso de puertos y resumen de direcciones); las copias están en `copias`. El entorno queda parado y los volúmenes `entorno-iot_*` conservan los datos: se vuelve a levantar con `.\desplegar.ps1`.